

FrostPuno: arquitectura distribuida y cómputo paralelo para la predicción de heladas mediante aprendizaje no supervisado en el altiplano de Puno

Mamani Mendoza, Joseph Elvis Pari Pari, Yimmy Ronaldo Quispe Galindo, Jahan Kevin
Flores Curasi, Héctor Luis Apaza Llanos, Yoel
Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional del Altiplano
Puno, Perú

Abstract—Frost events in the Peruvian highlands damage crops and cause neonatal mortality in alpaca and sheep herds, yet the same frost is the essential input for *chuño*, a freeze-dried potato that sustains regional food security. Producers currently decide when to protect crops or lay out potatoes based on personal experience, without integrated climate information. This work presents FrostPuno, a distributed system that groups the thermal regimes of thirteen districts in Puno using K-Means and delivers the result as a live production service. Three results are reported. First, the deployed system classifies all thirteen districts into three risk levels with a silhouette coefficient of 0.419 and a Davies-Bouldin index of 0.813, and the resulting territorial ranking matches regional geography without any hand-coded rule. Second, the system runs in production at zero infrastructure cost, reachable as a web application, a programming interface and an Android application, with automated retraining and a quality gate whose blocking behaviour is verified by tests. Third, an economic sustainability model is proposed, based on institutional licensing to local governments and cooperatives, showing that the operation reaches break-even with a small number of institutional subscribers. The parallel and distributed design decisions that make this operating cost possible are described: thread-pool ingestion with per-district fault isolation, four autonomous services coupled only through explicit contracts, and concurrent continuous-integration jobs.

Index Terms—parallel computing, distributed systems, unsupervised learning, K-Means, continuous integration, frost prediction, economic sustainability, high-Andean agriculture

Resumen—FrostPuno agrupa los regímenes térmicos de trece distritos de Puno mediante aprendizaje no supervisado y entrega el resultado como servicio en producción, accesible desde la web, una interfaz de programación y una aplicación Android. El sistema clasifica los trece distritos en tres niveles de riesgo con un coeficiente de silueta de 0.419, opera con costo de infraestructura nulo y se mantiene solo mediante reentrenamiento programado con una barrera de calidad verificada por pruebas. Se propone además un modelo de sostenibilidad económica basado en licenciamiento institucional que alcanza el punto de equilibrio con pocos suscriptores.

I. INTRODUCCIÓN

En la región Puno, por encima de los 3800 metros sobre el nivel del mar, las heladas presentan una doble condición. Dañan los cultivos y provocan mortalidad neonatal en crías de alpaca y ovino, patrimonio productivo de las familias altoandinas. Al mismo tiempo, constituyen el insumo indispensable del

chuño, papa deshidratada mediante congelamiento nocturno y secado diurno que se conserva durante años y forma parte de la seguridad alimentaria regional.

La decisión de proteger un cultivo, resguardar el ganado o tender la papa se toma habitualmente por experiencia empírica. No existe una herramienta que integre información climática abierta y la traduzca en recomendaciones territoriales. El problema no se limita al modelado: exige recolectar datos de múltiples ubicaciones, servirlos con baja latencia a usuarios dispersos y mantener el sistema operativo sin intervención constante.

Este artículo describe FrostPuno desde la perspectiva del cómputo paralelo y distribuido. Las contribuciones son las siguientes:

1. Una estrategia de ingesta paralela por hilos con aislamiento de fallos por ubicación, adecuada a una carga dominada por latencia de red.
2. Una arquitectura distribuida en cuatro servicios autónomos, desplegados en proveedores distintos y acoplados únicamente por contratos explícitos.
3. Un esquema de mantenimiento automatizado del componente inteligente, ejecutado como trabajos concurrentes de integración continua y protegido por una barrera de calidad verificada mediante pruebas.

II. TRABAJOS RELACIONADOS

La predicción de heladas se ha abordado tradicionalmente con modelos supervisados que requieren registros históricos etiquetados de eventos ocurridos. En territorios donde esos registros no existen o no son accesibles de forma programática, el agrupamiento ofrece una alternativa: permite descubrir regímenes climáticos sin necesidad de etiquetas.

En cuanto a la operación de sistemas con componentes de aprendizaje, la práctica de separar el entrenamiento por lotes de la inferencia en línea, y de intermediar ambos mediante un registro de modelos, se ha consolidado como patrón de referencia. FrostPuno adopta ese patrón y lo complementa con una barrera de calidad que impide promover modelos degradados, aspecto que suele quedar fuera de los trabajos académicos de alcance similar.

III. ARQUITECTURA DISTRIBUIDA

El sistema se descompone en cuatro servicios autónomos que se comunican por HTTP siguiendo contratos REST estables. Cada uno reside en una plataforma distinta, de modo que la distribución es efectiva y no una separación lógica dentro de un mismo proceso.

Cuadro I
DISTRIBUCIÓN DE LOS COMPONENTES DEL SISTEMA

Componente	Plataforma	Responsabilidad
Cliente Flutter	Vercel, APK	Presentación, ubicación
Servicio FastAPI	Render	Inferencia y reglas
Base de datos	Supabase	Persistencia
Fuente climática	Open-Meteo	Datos externos

El desacoplamiento se sostiene en dos decisiones. La primera es que el cliente conoce únicamente la dirección del servicio, inyectada en tiempo de compilación, por lo que el mismo código fuente produce la versión web, la versión de desarrollo y la aplicación Android. La segunda es que el servicio organiza su código en capas sucesivas de rutas, servicios y repositorios, lo que permite sustituir la persistencia sin alterar los controladores.

health	GET /health Health Check
ml	GET /ml/model-info Model Info
prediction	POST /predict/frost-risk Predict Frost Risk
predictions	GET /predictions/history Prediction History
weather	GET /weather/current Current Weather
Schemas	
CurrentWeatherResponse	Expand all object
FrostPredictionHistoryItem	Expand all object
FrostRiskRequest	Expand all object
FrostRiskResponse	Expand all object
HTTPValidationError	Expand all object
HealthResponse	Expand all object
ModelInfoResponse	Expand all object
ValidationError	Expand all object

Figura 1. Contrato REST publicado por el servicio en producción.

IV. PARALELISMO EN LA INGESTA DE DATOS

La construcción del conjunto de datos requiere recolectar series climáticas horarias para trece distritos. Cada solicitud es independiente y su costo está dominado por la latencia de red, no por el procesador, de modo que la carga es adecuada para paralelismo por hilos.

El listado 1 muestra la implementación. Se lanzan tantas tareas como distritos y se recolectan conforme finalizan, sin respetar el orden de emisión.

Listing 1. Ingesta paralela con aislamiento de fallos

```
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    futures = {
        executor.submit(fetch_weather_for_location,
                        record, start_date, end_date): record
        for record in records
    }
    for future in as_completed(futures):
        record = futures[future]
        try:
            frames.append(future.result())
        except Exception as exc:
            failures.append(
                f"{record['ubigeo']} {record['distrito']}: {exc}"
            )
```

Se destacan cuatro propiedades. La primera es el patrón de dispersión y recolección: la construcción del diccionario de tareas dispersa el trabajo y la iteración sobre las tareas completadas lo recolecta a medida que concluye. La segunda es el aislamiento de fallos: la captura de excepciones se realiza por tarea, de manera que la indisponibilidad de un distrito no interrumpe la recolección de los restantes y queda registrada de forma individual. La tercera es que el grado de paralelismo es configurable mediante un parámetro, ajustable según el ancho de banda disponible y las restricciones del proveedor de datos. La cuarta se refiere al rendimiento: siendo la carga dependiente de entrada y salida, el tiempo total tiende al de una sola solicitud en lugar del producto del número de distritos por ese tiempo, comportamiento consistente con la fracción paralelizable descrita por la ley de Amdahl.

V. DISTRIBUCIÓN DEL CÓMPUTO DE APRENDIZAJE

El sistema separa el cómputo intensivo del cómputo sensible a la latencia, siguiendo el patrón habitual en sistemas distribuidos de aprendizaje automático.

El entrenamiento se ejecuta por lotes y fuera de línea. Recorre una búsqueda exhaustiva de veinticuatro combinaciones de hiperparámetros y serializa el resultado en un registro de modelos junto con sus metadatos. La inferencia, en cambio, ocurre en línea: el servicio carga ese mismo artefacto una única vez por proceso y, ante cada solicitud, asigna el vector climático al centroide más próximo, operación de costo proporcional al número de grupos y por tanto muy económica.

La pieza que articula ambos extremos es la lista de características declarada en los metadatos. El servicio construye el vector de entrada siguiendo ese orden, por lo que un reentrenamiento no obliga a modificar el código del servidor. Este contrato constituye la frontera entre el subsistema de aprendizaje y el subsistema de servicio.

Adicionalmente, el servicio incorpora tres mecanismos de eficiencia. Emplea un servidor concurrente que atiende múltiples solicitudes simultáneas; mantiene una caché en memoria indexada por coordenada con tiempo de expiración, lo que evita consultar la fuente externa de forma repetida; e instancia una sola vez por proceso el registro de modelos, el servicio de predicción y el proveedor climático, evitando recargar el artefacto serializado en cada solicitud.

VI. MODELO DE AGRUPAMIENTO

VI-A. Conjunto de datos y características

La fuente climática es Open-Meteo, de la que se obtienen series horarias para trece distritos, resultando un conjunto de 4368 registros. La información territorial procede de una semilla compatible con el Instituto Nacional de Estadística e Informática.

Se partió de ocho variables climáticas. Un experimento de subconjuntos evidenció que la precipitación, la velocidad del viento, la nubosidad y la temperatura aparente introducían ruido o resultaban colineales, degradando la separación entre grupos. Al retirarlas, el coeficiente de silueta ascendió de 0.286 a 0.419, una mejora cercana al 47 por ciento obtenida mediante reducción y no mediante incorporación de información. Las cuatro características conservadas son la altitud estimada, la temperatura, el punto de rocío y la humedad relativa.

VI-B. Optimización de hiperparámetros

El algoritmo es K-Means, precedido por un escalado estandarizado necesario dado que las variables presentan magnitudes heterogéneas. La configuración se determinó mediante búsqueda exhaustiva que maximiza el coeficiente de silueta.

Cuadro II
ESPACIO DE BÚSQUEDA Y CONFIGURACIÓN SELECCIONADA

Hiperparámetro	Valores	Elegido
Número de grupos	3 a 8	3
Inicialización	k-means++, aleatoria	aleatoria
Reinicios	10, 25	10
Semilla	fija	42

VI-C. Evaluación

Al tratarse de aprendizaje no supervisado no se dispone de etiquetas de referencia, por lo que no procede el uso de exactitud ni de la medida F1. La evaluación cuantifica la calidad de la estructura descubierta.

Cuadro III
SELECCIÓN DEL NÚMERO DE GRUPOS

Grupos	Silueta	Davies-Bouldin
3 (elegido)	0.4195	0.813
4	0.3894	0.906
5	0.3760	0.820
6	0.3697	0.870
7	0.3645	0.865
8	0.3391	0.883

La configuración de tres grupos resulta superior en ambas medidas y coincide con los tres niveles de riesgo que requiere la aplicación. La asignación de niveles se realiza con posterioridad al entrenamiento, ordenando los grupos por su temperatura media: el más frío corresponde al riesgo alto. Se trata de una descripción de los grupos hallados y no de

una supervisión, dado que el entrenamiento no accede a esas etiquetas.

El resultado admite interpretación geográfica. El distrito de Macusani, situado a 4315 metros, queda clasificado en riesgo alto, mientras que Sandia, ubicado en valle a 2170 metros, corresponde al riesgo bajo. Esta correspondencia con la geografía regional emerge del agrupamiento y no fue codificada mediante reglas.



Figura 2. Agrupamiento de distritos con las métricas del modelo en producción.

VII. MANTENIMIENTO E INTEGRACIÓN CONTINUA

El mantenimiento de un sistema con componentes de aprendizaje no se limita al código. Comprende el reentrenamiento, el registro de versiones, el almacenamiento de características y el control de calidad previo a la promoción de un modelo.

VII-A. Trabajos concurrentes

Se implementaron cinco flujos independientes que se ejecutan de forma simultánea en entornos aislados ante cada incorporación de cambios. El primero entrena el modelo y publica los artefactos; el segundo aplica la barrera de calidad y ejecuta las pruebas de mantenimiento; el tercero verifica el servicio; el cuarto valida los conjuntos de datos; y el quinto analiza y compila el cliente. Al no existir dependencias entre ellos, el tiempo total corresponde al del flujo más lento y no a la suma de todos.

VII-B. Reentrenamiento adaptativo

El flujo de entrenamiento cuenta con una programación semanal, por lo que el modelo incorpora datos recientes sin

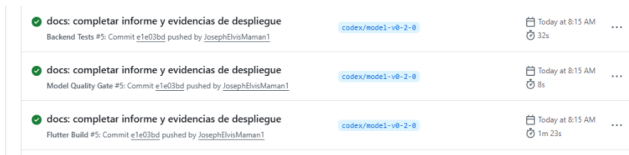


Figura 3. Ejecuciones concurrentes de los flujos de integración continua.

intervención humana. El procedimiento no se limita a repetir el entrenamiento: si la calidad obtenida no alcanza el objetivo definido, amplía la ventana de datos históricos de catorce a veintinueve y luego a cuarenta y cuatro días, y reintenta. Cada ejecución se registra en un historial que documenta la evolución de la calidad frente al volumen de datos disponible.

VII-C. Barrera de calidad

El reentrenamiento automático introduce el riesgo de promover un modelo inferior al vigente. Para evitarlo se verifica el coeficiente de silueta antes de cualquier promoción; si el valor se sitúa por debajo de 0.35, el proceso finaliza con código de error y la integración se detiene.

La existencia de la barrera no demuestra por sí sola su funcionamiento. Se implementaron seis pruebas automatizadas que se ejecutan dentro de la integración continua y verifican el ciclo completo: la generación de artefactos, la capacidad de carga e inferencia del modelo serializado, la cobertura territorial, la documentación de la búsqueda de hiperparámetros, la aprobación del modelo vigente y, de manera particular, el rechazo de un modelo deliberadamente degradado. Esta última prueba construye un conjunto de metadatos con una medida insuficiente y comprueba que el control lo bloquea.

VIII. RESULTADOS

VIII-A. Resultado territorial

El sistema clasifica los trece distritos evaluados en tres niveles de riesgo. El ordenamiento coincide con la geografía regional sin haber sido codificado: Macusani, a 4315 metros, resulta de riesgo alto, mientras que Sandia, ubicado en valle a 2170 metros, resulta de riesgo bajo. Esa correspondencia es el resultado más relevante del trabajo, porque indica que la estructura descubierta por el agrupamiento tiene sentido físico y no es un artefacto del procedimiento.

Cuadro IV
CLASIFICACIÓN TERRITORIAL OBTENIDA

Nivel	Distritos	Temp. media	Altitud media
Alto	6	4.0 °C	3905 m
Medio	6	11.6 °C	3893 m
Bajo	1	13.6 °C	2170 m

VIII-B. Producto en operación

El sistema no es un prototipo de laboratorio: opera de forma pública y continua. La tabla V resume las verificaciones.

Cuadro V
VERIFICACIONES SOBRE EL SISTEMA DESPLEGADO

Verificación	Resultado
Canales de acceso	Web, API y Android
Costo de infraestructura	0 USD mensuales
Pruebas del servicio	12 satisfactorias
Pruebas de mantenimiento	6 satisfactorias
Barrera de calidad	Aprobada, 0.419
Intervención humana requerida	Ninguna
Modelo en producción	K-Means v1.0.0

Dos cifras de esa tabla concentran el valor práctico del diseño. La primera es el **costo cero** de operación, consecuencia directa de haber distribuido el sistema en servicios que caben en los planes gratuitos de cada proveedor. La segunda es que el sistema **no requiere intervención humana**: el reentrenamiento ocurre por programación y la barrera de calidad decide si el modelo nuevo se promueve. Ambas condiciones son las que hacen viable el modelo económico que se describe en la sección siguiente.

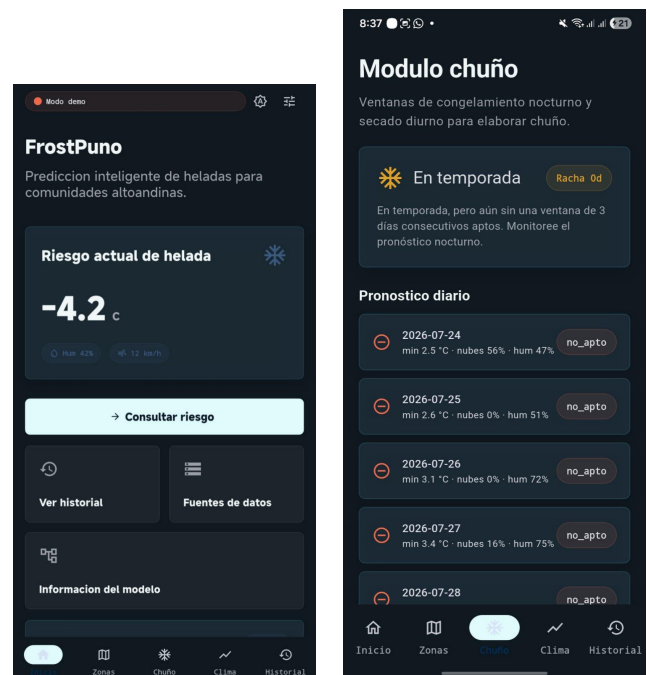


Figura 4. Cliente en operación: alerta diaria y módulo estacional de chuño.

VIII-C. Funcionalidad entregada al usuario

La aplicación traduce la salida del modelo en decisiones concretas: alerta diaria de helada con umbral configurable por el propio usuario, aviso diferenciado para ganado orientado a la protección de crías, detección de eventos inusuales frente a la climatología reciente, agrupamiento territorial, módulo estacional de chuño e histórico climático. Incorpora selección de idioma entre español e inglés.

IX. SOSTENIBILIDAD ECONÓMICA

Un sistema desplegado solo permanece útil si puede sostenerse en el tiempo. Esta sección presenta la estructura de costos verificada y un modelo de ingresos proyectado. Se distingue explícitamente entre ambos: los costos corresponden a la operación actual y son verificables; los ingresos son una proyección sujeta a las suposiciones que se declaran.

IX-A. Estructura de costos verificada

La decisión de distribuir el sistema en servicios pequeños y autónomos tiene una consecuencia económica directa: cada componente cabe en el plan gratuito de su proveedor. El costo operativo actual es nulo.

Cuadro VI
COSTO MENSUAL DE OPERACIÓN (USD APROXIMADOS)

Componente	Actual	Plan pagado
Servicio backend	0	7
Cliente web	0	20
Base de datos	0	25
Integración continua	0	0
Fuente climática	0	0
Total	0	52

El escenario de plan pagado corresponde a precios públicos aproximados de los proveedores y sería necesario únicamente al superar los límites del nivel gratuito, principalmente por volumen de usuarios concurrentes. La ausencia de servidores propios elimina además el costo de administración de infraestructura, que en un despliegue tradicional suele superar al costo del propio servicio.

IX-B. Modelo de ingresos propuesto

El usuario final directo es el productor altoandino, para quien el pago individual no es viable ni deseable. El modelo propuesto mantiene la aplicación gratuita para el productor y traslada el ingreso a las instituciones que se benefician de la información agregada.

Cuadro VII
LÍNEAS DE INGRESO PROPUESTAS

Línea	Cliente	Propuesta de valor
Licencia institucional	Municipalidades y gobierno regional	Panel de riesgo por distrito para dirigir campañas de prevención
Suscripción asociativa	Cooperativas y asociaciones de productores	Alertas y ventanas de chuño para sus asociados
Servicio de datos	Aseguradoras agrarias	Historial de riesgo territorial para tarificación
Convenio	Entidades públicas del sector	Cobertura ampliada y validación de campo

IX-C. Punto de equilibrio

Bajo el escenario de plan pagado de la tabla VI, el costo operativo anual asciende a unos 624 dólares. Suponiendo una licencia institucional anual de 500 dólares por municipalidad, cifra deliberadamente conservadora frente al presupuesto de una campaña de prevención de heladas, el punto de equilibrio se alcanza con **dos** entidades suscritas. La provincia de Puno agrupa más de una decena de municipalidades distritales, de modo que el umbral se sitúa muy por debajo del mercado potencial inmediato.

El costo marginal de incorporar una entidad adicional es prácticamente nulo, porque el sistema ya calcula el agrupamiento de todos los distritos en la misma ejecución. Esa asimetría entre costo fijo bajo y costo marginal nulo es la que otorga viabilidad al modelo.

IX-D. Condiciones para la rentabilidad

Tres decisiones técnicas del trabajo son las que sostienen esta viabilidad y conviene señalarlas de forma explícita:

- La **arquitectura distribuida** permite que cada componente permanezca dentro de un plan gratuito, lo que reduce el costo fijo a cero durante la etapa de validación.
- El **mantenimiento automatizado** elimina el costo recurrente de personal dedicado a reentrenar y verificar el modelo, que en un sistema de aprendizaje suele ser el gasto dominante.
- La **ingesta paralela** mantiene el tiempo de actualización acotado al incorporar nuevos distritos, de modo que ampliar la cobertura territorial no incrementa proporcionalmente el costo de cómputo.

Debe señalarse que este modelo no ha sido validado comercialmente: no existen contratos firmados ni ingresos percibidos. Constituye una proyección fundamentada en la estructura de costos real del sistema y en la identificación de actores con capacidad de pago, y su verificación es parte del trabajo futuro.

X. LIMITACIONES

Los niveles de riesgo se derivan del perfil térmico de cada grupo y no de un registro oficial de heladas observadas, dado que no existe un conjunto etiquetado disponible para estos distritos. El Servicio Nacional de Meteorología e Hidrología se declara como fuente oficial prioritaria en el sistema, si bien no expone una interfaz pública estable, por lo que Open-Meteo constituye la fuente operativa. La semilla territorial corresponde a una versión reducida. Los umbrales aplicados al chuño y al ganado responden a un diseño documentado con fuentes, sin constituir cifras oficiales. La detección de eventos inusuales emplea climatología de treinta días y no una serie multianual. Finalmente, el plan gratuito de la plataforma de alojamiento suspende la instancia tras un periodo de inactividad, lo que introduce demora en la primera solicitud.

XI. CONCLUSIONES

Se presentó un sistema distribuido para la predicción de heladas en el altiplano de Puno, desplegado en producción y accesible como aplicación web, interfaz de programación y aplicación Android.

Desde la perspectiva del cómputo paralelo y distribuido, el trabajo aporta tres elementos. La ingesta climática se paraleliza mediante un conjunto de hilos con aislamiento de fallos por ubicación, aprovechando que la carga depende de la latencia de red. La arquitectura se distribuye en cuatro servicios autónomos alojados en proveedores distintos, acoplados únicamente por contratos explícitos, entre los que destaca la lista de características que separa el subsistema de aprendizaje del de servicio. El mantenimiento se automatiza como trabajos concurrentes de integración continua, con reentrenamiento programado de ventana adaptativa y una barrera de calidad cuyo funcionamiento se verifica mediante pruebas.

El modelo de agrupamiento alcanza un coeficiente de silueta de 0.419 y un índice de Davies-Bouldin de 0.813, y produce una clasificación territorial coherente con la geografía regional sin haber sido programada explícitamente.

Como trabajo futuro se plantea la incorporación de observaciones oficiales como validación cruzada del agrupamiento, la ampliación de la cobertura territorial mediante la exportación completa de distritos y la evolución de la ingesta paralela hacia un esquema distribuido en varios nodos.

DISPONIBILIDAD DEL CÓDIGO

El código fuente completo se encuentra disponible de forma pública en <https://github.com/JosephElvisMaman1/frost-puno>. La aplicación web opera en <https://frost-puno.vercel.app> y el servicio en <https://frost-puno.onrender.com>, cuya documentación interactiva reside en la ruta /docs. La aplicación Android se distribuye como archivo app-release.apk generado desde el repositorio.

APÉNDICE A

FRAGMENTOS REPRESENTATIVOS DEL CÓDIGO

Se incluyen los fragmentos centrales de los aportes descritos. El listado 2 corresponde a la búsqueda de hiperparámetros del modelo de agrupamiento.

Listing 2. Búsqueda exhaustiva de hiperparámetros

```
for k in k_range:
    for init in init_options:
        for n_init in n_init_options:
            pipeline = build_pipeline(k, init=init, n_init=
                n_init)
            labels = pipeline.fit_predict(features)
            scaled = pipeline.named_steps["scaler"].transform(
                features)
            metrics = {
                "silhouette": float(silhouette_score(scaled,
                    labels)),
                "davies_bouldin": float(davies_bouldin_score(
                    scaled, labels)),
                "inertia": float(pipeline.named_steps["kmeans"].
                    inertia_),
            }
            grid.append({"k": k, "init": init,
                "n_init": n_init, **metrics})

best = max(grid, key=lambda row: row["silhouette"])
```

El listado 3 presenta la barrera de calidad que detiene la integración ante una regresión del modelo.

Listing 3. Barrera de calidad del modelo

```
def run(metadata_path, min_silhouette):
    silhouette = load_silhouette(metadata_path)
    print(f"Cluster quality gate: silhouette={silhouette:.6f}
        }, "
        f"minimum={min_silhouette:.6f}")
    if silhouette < min_silhouette:
        print("QUALITY GATE FAILED")
        return 1
    print("QUALITY GATE PASSED")
    return 0
```

El listado 4 corresponde a la prueba que verifica el funcionamiento de dicha barrera.

Listing 4. Verificación del bloqueo de un modelo degradado

```
def test_quality_gate_blocks_degraded_model(tmp_path,
    metadata):
    degraded = dict(metadata)
    degraded["metrics"] = {"silhouette": 0.10,
        "davies_bouldin": 3.0, "inertia": 1.0}
    bad_path = tmp_path / "cluster_metadata.json"
    bad_path.write_text(json.dumps(degraded), encoding="utf-8")

    assert run_gate(bad_path, min_silhouette=0.35) == 1
```

APÉNDICE B

REPRODUCCIÓN DE LOS RESULTADOS

Los siguientes comandos permiten reproducir el entrenamiento, la barrera de calidad y las pruebas descritas.

Listing 5. Comandos de reproducción

```
python -m ml_pipeline.clustering.train_clusters
python -m ml_pipeline.registry.check_cluster_quality --min-
    silhouette 0.35
python -m pytest ml_pipeline/tests/test_maintenance_ci.py -v
python -m ml_pipeline.clustering.adaptive_retrain --target
    0.45 --max-iters 3
pytest backend_fastapi/tests -q
```